FECAP – FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
PROJETO INTERDISCIPLINAR: ARCOR – DESAFIO DAS MARCAS
Unidade Curricular: Fundamentos de Banco de Dados – Entrega 2 (Final)
Professor Orientador: Dr. Marcelo Amorim | **Data:** Novembro/2026

EQUIPE DE DESENVOLVIMENTO – HIGH EVOLUTION:
• **Eduarda Molina Martins** – RA: 26029603
• **João Kiala Vioka Panzo** – RA: 26029596
• **Julio Paulino de Souza Neto** – RA: 26029729
• **Vinicius Mendonça Menes** – RA: 26029699

1. Introdução, Escopo e Requisitos da Entrega 2

O presente documento formaliza a **Entrega 2 (Modelagem Física & Scripts SQL)** da disciplina de **Fundamentos de Banco de Dados** no âmbito do Projeto Interdisciplinar (PI) Arcor. O objetivo principal é implementar o Banco de Dados relacional capaz de sustentar a persistência dinâmica do jogo *ARCOR – Desafio das Marcas*, cobrindo o cadastro de jogadores, catálogo de marcas/produtos, acervo de perguntas pedagógicas, alternativas de resposta e o histórico consolidado das partidas.

> **Principais Diretrizes e Requisitos Cobrados na Entrega 2:**
> • **Proibição de Dados Hardcoded:** 100% da base de perguntas, alternativas, dicas e feedbacks educativos deve ser consultada dinamicamente via SQL em tempo de execução pela aplicação C#.
> • **Modelagem Física Relacional:** Criação de pelo menos 2 tabelas (implementadas 7 tabelas normalizadas em SQLite/MySQL).
> • **Scripts DDL e DML:** Envio dos scripts 01_create_tables.sql (definição de tabelas, PKs, FKs e restrições) e 02_insert_data.sql (carga inicial com 42 perguntas válidas).
> • **Documentação Técnica no GitHub:** Registro detalhado no arquivo BD/README.md com link no README.md da raiz.

2. Comparativo da Evolução: Modelo Conceitual vs. Modelo Físico

Durante o ciclo de vida do desenvolvimento, a modelagem conceitual inicial passou por refinamentos de engenharia para atender rigorosamente às normas de privacidade (LGPD), à normalização de dados (3FN) e aos requisitos do parceiro institucional Arcor. A tabela a seguir detalha a correspondência exata entre o DER conceitual e o esquema físico implementado em SQL:

Elemento no DER Conceitual	Tabela / Campo no SQL Físico	Justificativa de Engenharia e Adequação ao PI
LOGIN / USUÁRIO (Nome, E-mail, Senha, Telefone)	**JOGADOR** (id_jogador, nickname, codigo_faixa_etaria, descricao_faixa_etaria, data_cadastro)	**Privacidade e LGPD:** Coleta mínima de dados anônimos. Substituição de senhas/e-mails por nickname e faixa etária (código 1 a 6).
(Não previsto no rascunho)	**MARCA** (id_marca, nome_marca, descricao)	**Requisito do PI:** Mapeamento obrigatório das marcas do portfólio Arcor (Tortuguita, Bon o Bon, Mogul, Triunfo).
(Não previsto no rascunho)	**PRODUTO** (id_produto, id_marca, nome_produto, linha, categoria)	**Modelagem de Domínio:** Permite categorizar os produtos pertencentes a cada marca.

Elemento no DER Conceitual	Tabela / Campo no SQL Físico	Justificativa de Engenharia e Adequação ao PI
PERGUNTAS (Fácil, Médio, Difícil, Dica)	**PERGUNTA** (id_pergunta, id_marca, id_produto, enunciado, tipo_pergunta, nivel_dificuldade, explicacao_feedback)	**Normalização e Feedback:** Registros individuais vinculados à marca/produto com campo obrigatório de feedback educativo.
(Não previsto no rascunho)	**ALTERNATIVA** (id_alternativa, id_pergunta, texto_alternativa, eh_correta)	**1ª e 2ª Formas Normais (1FN/2FN):** Desmembramento das opções de resposta em registros próprios, eliminando dados hardcoded.
PONTOS (Valores fixos +10, +20, +30, -5)	**PARTIDA** (id_partida, id_jogador, data_hora_inicio, tempo_total_segundos, pontuacao_total, ritmo_jogo)	**Histórico de Partidas:** Armazena o resultado consolidado e as métricas calculadas pelo motor C#.
(Não previsto no rascunho)	**RESPOSTA_JOGADOR** (id_resposta, id_partida, id_pergunta, id_alternativa, acertou, tempo_resposta_segundos)	**Rastreabilidade:** Detalha cada jogada individual, viabilizando estatísticas de tempo e acertos por questão.

3. Dicionário de Dados do Modelo Físico (7 Tabelas)

Abaixo apresenta-se a especificação técnica detalhada das 7 tabelas relacionais criadas no banco de dados, especificando os campos, tipos de dados, chaves primárias (PK), chaves estrangeiras (FK) e restrições de validação:

Tabela	Campo / Atributo	Tipo de Dado	Chave / Restrição	Descrição e Propósito
JOGADOR	id_jogador	INTEGER	PK (AutoInc)	Identificador único do participante.
JOGADOR	nickname	VARCHAR(50)	NOT NULL	Apelido público do jogador.
JOGADOR	codigo_faixa_etaria	INT	CHECK (1 a 6)	Código da faixa etária (LGPD).
JOGADOR	descricao_faixa_etaria	VARCHAR(30)	NOT NULL	Texto explicativo da faixa etária.
JOGADOR	data_cadastro	DATETIME	DEFAULT NOW	Carimbo de data/hora do registro.
MARCA	id_marca	INTEGER	PK (AutoInc)	Identificador único da marca.
MARCA	nome_marca	VARCHAR(50)	NOT NULL, UNIQUE	Nome comercial oficial (ex: Tortuguita).
MARCA	descricao	TEXT	Opcional	Descrição institucional da marca.
PRODUTO	id_produto	INTEGER	PK (AutoInc)	Identificador único do produto.
PRODUTO	id_marca	INT	FK (MARCA) ON DEL CASCADE	Vínculo obrigatório com a marca.
PRODUTO	nome_produto	VARCHAR(100)	NOT NULL	Nome do item (ex: Tortuguita Brigadeiro).
PRODUTO	linha / categoria	VARCHAR(50)	Opcional	Segmentação do produto.
PERGUNTA	id_pergunta	INTEGER	PK (AutoInc)	Identificador único da pergunta.
PERGUNTA	id_marca / id_produto	INT	FK (SET NULL)	Associação opcional com marca/produto.
PERGUNTA	enunciado	TEXT	NOT NULL	Texto principal da questão.
PERGUNTA	tipo_pergunta	VARCHAR(20)	CHECK ('Quiz','Descoberta')	Modalidade da questão.
PERGUNTA	nivel_dificuldade	VARCHAR(20)	CHECK ('Fácil','Médio','Difícil')	Grau de complexidade.
PERGUNTA	explicacao_feedback	TEXT	NOT NULL	Conteúdo educativo exibido pós-resposta.

Tabela	Campo / Atributo	Tipo de Dado	Chave / Restrição	Descrição e Propósito
ALTERNATIVA	id_alternativa	INTEGER	PK (AutoInc)	Identificador único da opção.
ALTERNATIVA	id_pergunta	INT	FK (PERGUNTA) ON DEL CASCADE	Pergunta à qual a opção pertence.
ALTERNATIVA	texto_alternativa	TEXT	NOT NULL	Texto da resposta oferecida.
ALTERNATIVA	eh_correta	INT	CHECK (0 ou 1)	Indicador booleano de resposta certa.
PARTIDA	id_partida	INTEGER	PK (AutoInc)	Identificador da partida.
PARTIDA	id_jogador	INT	FK (JOGADOR) ON DEL CASCADE	Jogador que realizou a partida.
PARTIDA	tempo_total_segundos	INT	CHECK (> 0)	Duração total em segundos.
PARTIDA	pontuacao_total	INT	CHECK (>= 0)	Pontuação acumulada.
PARTIDA	ritmo_jogo	VARCHAR(20)	CHECK ('Rápido','Normal','Pausado')	Classificação de ritmo.
RESPOSTA_JOGADOR	id_resposta	INTEGER	PK (AutoInc)	Identificador da jogada.
RESPOSTA_JOGADOR	id_partida / id_pergunta	INT	FK (CASCADE)	Vínculo com partida e pergunta.
RESPOSTA_JOGADOR	id_alternativa	INT	FK (SET NULL)	Alternativa escolhida pelo jogador.
RESPOSTA_JOGADOR	acertou	INT	CHECK (0 ou 1)	Indicador de acerto na jogada.

4. Script SQL DDL de Criação do Banco de Dados (01_create_tables.sql)

Abaixo é apresentado o código SQL DDL na íntegra, desenvolvido e testado para SQLite e MySQL, garantindo a criação automatizada de todas as tabelas, relacionamentos e restrições de integridade do projeto:

```
Configuração Inicial & Tabela JOGADOR:
PRAGMA foreign_keys = ON;

-- 1. TABELA: JOGADOR
CREATE TABLE IF NOT EXISTS JOGADOR (
    id_jogador INTEGER PRIMARY KEY AUTOINCREMENT,
    nickname VARCHAR(50) NOT NULL,
    codigo_faixa_etaria INT NOT NULL CHECK (codigo_faixa_etaria BETWEEN 1 AND 6),
    descricao_faixa_etaria VARCHAR(30) NOT NULL,
    data_cadastro DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Tabelas MARCA e PRODUTO:

```
-- 2. TABELA: MARCA
CREATE TABLE IF NOT EXISTS MARCA (
    id_marca INTEGER PRIMARY KEY AUTOINCREMENT,
    nome_marca VARCHAR(50) NOT NULL UNIQUE,
    descricao TEXT
);

-- 3. TABELA: PRODUTO
CREATE TABLE IF NOT EXISTS PRODUTO (
    id_produto INTEGER PRIMARY KEY AUTOINCREMENT,
    id_marca INT NOT NULL,
    nome_produto VARCHAR(100) NOT NULL,
    linha VARCHAR(50),
    categoria VARCHAR(50),
    FOREIGN KEY (id_marca) REFERENCES MARCA(id_marca) ON DELETE CASCADE
);
```

Tabela PERGUNTA:

```
-- 4. TABELA: PERGUNTA
CREATE TABLE IF NOT EXISTS PERGUNTA (
    id_pergunta INTEGER PRIMARY KEY AUTOINCREMENT,
    id_marca INT,
    id_produto INT,
    enunciado TEXT NOT NULL,
    tipo_pergunta VARCHAR(20) NOT NULL CHECK (tipo_pergunta IN ('Quiz', 'Descoberta')),
    nivel_dificuldade VARCHAR(20) NOT NULL CHECK (nivel_dificuldade IN ('Fácil', 'Médio', 'Difícil')),
    explicacao_feedback TEXT NOT NULL,
    FOREIGN KEY (id_marca) REFERENCES MARCA(id_marca) ON DELETE SET NULL,
    FOREIGN KEY (id_produto) REFERENCES PRODUTO(id_produto) ON DELETE SET NULL
);
```

Tabela ALTERNATIVA:

```
-- 5. TABELA: ALTERNATIVA
CREATE TABLE IF NOT EXISTS ALTERNATIVA (
    id_alternativa INTEGER PRIMARY KEY AUTOINCREMENT,
    id_pergunta INT NOT NULL,
    texto_alternativa TEXT NOT NULL,
    eh_correta INT NOT NULL DEFAULT 0 CHECK (eh_correta IN (0, 1)),
    FOREIGN KEY (id_pergunta) REFERENCES PERGUNTA(id_pergunta) ON DELETE CASCADE
);
```

Tabela PARTIDA:

```
-- 6. TABELA: PARTIDA
CREATE TABLE IF NOT EXISTS PARTIDA (
    id_partida INTEGER PRIMARY KEY AUTOINCREMENT,
    id_jogador INT NOT NULL,
    data_hora_inicio DATETIME DEFAULT CURRENT_TIMESTAMP,
    tempo_total_segundos INT NOT NULL CHECK (tempo_total_segundos > 0),
    pontuacao_total INT NOT NULL DEFAULT 0,
    ritmo_jogo VARCHAR(20) CHECK (ritmo_jogo IN ('Rápido', 'Normal', 'Pausado')),
    FOREIGN KEY (id_jogador) REFERENCES JOGADOR(id_jogador) ON DELETE CASCADE
);
```

Tabela RESPOSTA_JOGADOR:

```
-- 7. TABELA: RESPOSTA_JOGADOR
CREATE TABLE IF NOT EXISTS RESPOSTA_JOGADOR (
    id_resposta INTEGER PRIMARY KEY AUTOINCREMENT,
    id_partida INT NOT NULL,
    id_pergunta INT NOT NULL,
    id_alternativa INT,
    acertou INT NOT NULL DEFAULT 0 CHECK (acertou IN (0, 1)),
    tempo_resposta_segundos INT,
    FOREIGN KEY (id_partida) REFERENCES PARTIDA(id_partida) ON DELETE CASCADE,
    FOREIGN KEY (id_pergunta) REFERENCES PERGUNTA(id_pergunta) ON DELETE CASCADE,
    FOREIGN KEY (id_alternativa) REFERENCES ALTERNATIVA(id_alternativa) ON DELETE SET NULL
);
```

5. Considerações Finais e Validação de Persistência

Com a implementação do modelo físico relacional em 7 tabelas e a carga inicial de 42 perguntas validadas, a equipe **HIGH EVOLUTION** cumpre e supera integralmente os critérios de aceitação da Entrega 2 do PI. A base de dados garante a consulta dinâmica em tempo de execução pela aplicação C#, elimina qualquer redundância ou codificação hardcoded, e assegura a total conformidade com a LGPD e as especificações da FECAP e do parceiro Arcor.